

# Moved Object Detection Using DETR: A Feature-Difference Approach

## 1. Introduction & Objective

The objective of this project was to develop an end-to-end DETR pipeline to detect objects that moved between two frames of surveillance footage. We want to find objects with significant movement ( $\text{IoU} = 0$ ) between frame pairs.

## 2. Data Preparation & Pipeline

### Ground Truth Generation

The project utilized a modified subset of the VIRAT Video Dataset for training and evaluation. Ground truth annotations were generated using a semi-automated labeling pipeline based on feature similarity and centroid tracking. This process yielded a total of 290 annotated frame pairs (files), which were subsequently utilized for training and testing. The complete set of generated ground truth annotations has been archived and is accessible via [\[Link/Attachment\]](#)

### Preprocessing

The Dataset class in `dataload.py` implements a standard preprocessing pipeline to ensure compatibility with the DETR backbone:

**Resizing:** All input images (both "before" and "after" frames) are resized to a fixed resolution of  $800 \times 800$  pixels. This ensures consistent tensor shapes for batching. The ground truth bounding boxes are also rescaled proportionally to match the new dimensions.

**Normalization:** Inside the model's forward pass, images are normalized using the standard ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). Pixel values are first scaled to the  $[0,1]$  range

### Dataset Split

A 80-20 Train-Test split was implemented on the annotated dataset.

**Train-Validation Split:** 80% of the data was allocated for training and validation. Within this subset, a further 90-10 split (90% training, 10% validation) was applied to monitor overfitting during the training process.

**Test Split:** The remaining 20% of the data was held out exclusively for final evaluation and metrics reporting.

**Reproducibility:** All splits were generated using `torch.utils.data.random_split` with a fixed random seed (`torch.manual_seed(23)`) to guarantee deterministic results across runs.

## 3. Methodology & Architecture

### Selected Architecture: Feature-Difference Injection Strategy

A **Feature-Difference architecture** was implemented to leverage semantic changes directly.

**Backbone:** The frozen ResNet-50 backbone from `facebook/detr-resnet-50` was utilized.

**Feature Extraction Strategy:** Instead of manually truncating the model at an intermediate layer, a Forward Hook (`register_forward_hook`) was employed on the ResNet backbone.

### Mechanism:

The "Before" image ( $I_B$ ) and "After" image ( $I_A$ ) are passed through the backbone independently. Feature maps ( $F_B, F_A$ ) are extracted from the final convolutional block. The absolute difference between these feature maps is computed:  $\text{delta\_feat} = |F_B - F_A|$ . This difference tensor  $\text{delta\_feat}$  is then injected back into the model pipeline, replacing the standard feature map input for the Transformer Encoder.



Figure 1: Visualization of Feature Extraction and Difference Computation.

### Implementation Details

The implementation utilizes a custom DETR class (in `model.py`) inheriting from `torch.nn.Module`.

**Hook Logic:** A custom `feature_hook` is registered to intercept the output of the backbone.

**Injection:** During the forward pass, the difference is calculated as  $\text{delta\_feat} = \text{torch.abs}(\text{tens\_a} - \text{tens\_b})$ . This tensor is then wrapped in a `FakeFeat` class to mimic the internal data structure expected by the HuggingFace DETR implementation.

**Bypassing the Default Flow:** The `custom_features` attribute is used to override the backbone's natural output with the calculated  $\text{delta\_feat}$  immediately prior to the Transformer Encoder.

### Rationale for Architecture Choice

The Feature-Difference strategy was chosen over the Pixel-Difference baseline due to data efficiency and pre-training compatibility. Feeding pixel-difference images (often resembling gray noise) into a ResNet backbone trained on natural RGB images would require the model to learn low-level feature extraction from scratch, likely leading to overfitting on this small dataset. In contrast, the Feature-Difference approach allows the backbone to utilize its pre-trained "object recognition" capabilities first, computing semantic differences only in the high-level feature space.

## 4. Ablation Tests, Results, and Discussion

### Ablation Study Methodology

To evaluate the efficacy of different fine-tuning strategies within the DETR pipeline, two distinct ablation tests were conducted. These experiments were designed to isolate the impact of updating specific components of the Transformer architecture while keeping the ResNet-50 backbone frozen to preserve its pre-trained feature extraction capabilities.

1. **Experiment A (Heads Only):** In this test, the entire ResNet-50 backbone and the Transformer Encoder/Decoder blocks were frozen. Only the final Classification Head and Bounding Box Regressor were trained. This configuration resulted in a lightweight model with only **134,411 trainable parameters**.
2. **Experiment B (Full Transformer):** In this test, the ResNet-50 backbone remained frozen, but the entire Transformer (Encoder, Decoder, Input Projections, and Query Embeddings) was unfrozen and trained alongside the heads. This resulted in a much larger model with **18,048,011 trainable parameters**.

## Results

Both models were trained for 360 epochs using identical hyperparameters (Batch Size: 8, Learning Rate Strategy: StepLR) and the exact same 80-20 dataset split. Evaluation was performed using strict thresholds: a Confidence Threshold of 0.5 and an IoU Threshold of 0.5. These thresholds ensure that only high-confidence predictions with significant spatial overlap with the ground truth are counted as True Positives. All training runs were conducted on a single NVIDIA A100 GPU to ensure consistent benchmarking.

The performance metrics for both experiments are summarized below:

| Metric                   | Exp A: Heads Only   | Exp B: Transformer (Run D4) |
|--------------------------|---------------------|-----------------------------|
| Trainable Parameters     | 134,411             | 18,048,011                  |
| Training Time            | 7,545.5s (~2.1 hrs) | 7,799.6s (~2.2 hrs)         |
| Final Train Loss         | 1.3657              | 1.187                       |
| Final Val Loss           | 1.4688              | 1.3754                      |
| Precision                | 0.2278              | 0.2292                      |
| Recall                   | 0.5866              | 0.4764                      |
| F1-Score                 | 0.3282              | 0.3095                      |
| mAP (Mean Avg Precision) | 0.1244              | 0.0841                      |
| mAR (Mean Avg Recall)    | 0.2944              | 0.2543                      |

The full output file can be found in the output files (d3\_\*.out for Exp A and d4\_\*.out for Exp B) attached with the report. The loss curves are also attached.

## Discussion and Analysis

Experiment A (Heads Only) outperformed Experiment B (Full Transformer) on the Test set, achieving a ~48% higher mAP (0.1244 vs 0.0841) and ~16% higher mAR (0.2944 vs 0.2543).

**Evidence of Overfitting:** Experiment B exhibited signs of overfitting. It achieved significantly lower Training and Validation losses than Experiment A but failed to generalize to the Test set.

**Parameter Efficiency:** With over 100 times fewer trainable parameters (134k vs 18M), Experiment A demonstrated that the pre-trained Transformer features were already sufficient. Unfreezing these layers in Experiment B didn't achieve any significant improvement, on the contrary, worsened the result.

**Analysis of Low Metrics:** The overall low metrics (Precision ~ 0.23, Recall ~ 0.59) are primarily attributed to the small dataset size.



## 5. Output Visualization

From the ablation tests, it is clear that the model from Experiment A performed better. A few outputs from the model are visualized below (in Red) along with the Ground Truth (in Green). As some of the bounding boxes might overlap and block the output, prediction boxes are processed via Non-Maximum Suppression (NMS) with a strict threshold. Any box that overlaps with a higher-confidence detection by an IoU of more than 0.1 is suppressed/removed to eliminate redundant duplicates and ensure a clean visualization.





